```
making room
* for additional growth.  The over-allocation is mild, but is
* enough to give linear-time amortized behavior over a long
* sequence of appends() in the presence of a poorly-
performing
* system realloc().
* Add padding to make the allocated size multiple of 4.
* The growth pattern is:  0, 4, 8, 16, 24, 32, 40, 52, 64,
76, 88, 120, 160 ...
* Note: new_allocated won't overflow because the largest
possible value
*       is PY_SSIZE_T_MAX * (9 / 8) + 6 which always fits in a
size_t.
     */
    new_allocated = ((size_t)newsize + (newsize >> 3) + 6) &
~(size_t)3;
    /* Do not overallocate if the new size is closer to
overalocated size
     * than to the old size.
     */
'''
###   eg: if the current size is 24
###   ((size_t)newsize + (newsize >> 3) + 6) == increases the
one eighth which is 15 so 24+3 => 24+6 = 30
###   ~(size_t)3 = -4
###   30 & -4 = 32

### Note: The calculations vary based on the size of the
object used in the list
```

Empty list

When an empty list is created, it will always point to a different address. It will also hold preallocated memory as well.

```
# When creating two empty list it will be point different
address space
a = []
b = []
if a is not b :
  print("A and B are not mapped to same address")
  print("address of a: ", id(a))
  print("address of b: ", id(b))

print("A size of list: ", sys.getsizeof(a))
print("B size of list: ", sys.getsizeof(b))
```

The output is:

```
A and B are not mapped to same address
address of a:  140509666702080
address of b:  140509666510912
A size of list:  56
B size of list:  56
```

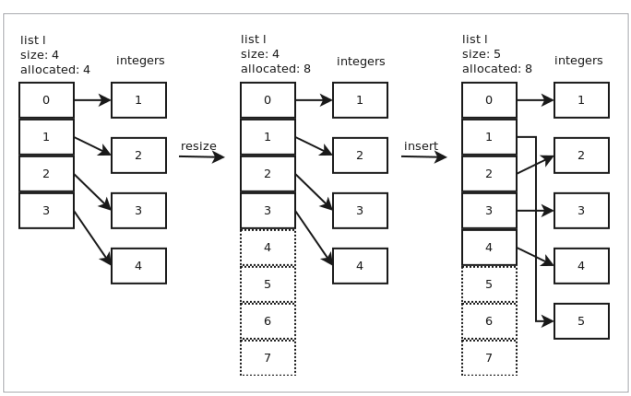

Figure 1: Memory allocation in list
(Credits: *https://www.laurentluce.com/images/blog/list/list_insert.png)*

Removal and insertion

When we perform removal, the allocated memory will shrink without changing the address of the variable. While performing insert, the allocated memory will expand and the address might get changed as well.

```
# pop an element from the between of the array
print("address before change: ", id(list1))
print("Remove element from list", list1.pop(2))
print("Size of list after removal: ", sys.getsizeof(list1))
print("Remove element from list", list1.pop(2))
print("Size of list after removal: ", sys.getsizeof(list1))
print("Remove element from list", list1.pop(2))
print("Size of list after removal: ", sys.getsizeof(list1))
print("Remove element from list", list1.pop(2))
print("Size of list after removal: ", sys.getsizeof(list1))
print("Remove element from list", list1.pop(2))
print("Size of list after removal: ", sys.getsizeof(list1))
print(list1)
print("address after change: ", id(list1))
```

The output is:

```
address before change:  140509666477824
Remove element from list three
Size of list after removal:  192
Remove element from list 4
Size of list after removal:  192
Remove element from list 5
Size of list after removal:  192
Remove element from list 6
Size of list after removal:  192
Remove element from list 7
Size of list after removal:  136
[100, 2, 8, 9, 10, 11, 12]
```